

Python Topic 17 — Generators & Iterators

Full Stack Python + AI Roadmap • Taught in 3 layers: New to Python → Intermediate → Expert • Powers streaming LLM responses

Layer 1 — New to Python

A **generator** produces values one at a time, only when you ask — instead of building them all at once and holding them in memory.

```
# Regular function → returns everything at once
def get_numbers():
    return [1, 2, 3, 4, 5]    # builds the whole list in memory

# Generator → hands you one value at a time using yield
def get_numbers():
    yield 1
    yield 2
    yield 3

for n in get_numbers():
    print(n)    # 1, 2, 3 → produced one at a time
```

The magic word is **yield**. A function with **yield** becomes a generator — it pauses after each **yield** and resumes where it left off when you ask for the next value.

Layer 2 — Intermediate (real code + how it works)

return vs **yield**

```
# return → gives back ONE result, function ends
def normal():
    return 1
    return 2    # never runs – return exits immediately

# yield → gives back MANY results, function pauses & resumes
def generator():
    yield 1    # pause here, hand back 1
    yield 2    # resume here next time, hand back 2
    yield 3
```

How a generator actually runs — it's lazy

```
def count_up():
    print("start")
    yield 1
    print("resumed")
    yield 2
    print("resumed again")
    yield 3

gen = count_up()      # nothing prints yet! generator not started
next(gen)             # "start" → returns 1 (runs until first yield, pauses)
next(gen)             # "resumed" → returns 2
next(gen)             # "resumed again" → returns 3
next(gen)             # ❌ StopIteration – nothing left
```

i Mental model: the function **freezes** at each **yield** and thaws on the next **next()** call — remembering all its local state.

Generator expressions — lazy comprehensions (round brackets)

```
squares_list = [n**2 for n in range(1000000)] # builds ALL million in memory
squares_gen  = (n**2 for n in range(1000000)) # builds NONE yet – lazy!
```

Just swap **[]** for **()** and you get a generator instead of a list. Huge memory savings for large data.

Why generators matter — memory

```
# ❌ loads the ENTIRE file into memory (crashes on a 10GB file)
lines = open("huge.txt").readlines()

# ✅ reads one line at a time (handles any size)
def read_lines(path):
    with open(path) as f:
        for line in f:
            yield line.strip()

for line in read_lines("huge.txt"): # constant memory, any file size
    process(line)
```

Layer 3 — Expert (internals & gotchas)

1. Generators are single-use

```
gen = (n for n in range(3))
list(gen)    # [0, 1, 2]
list(gen)    # [] ← empty! generator is EXHAUSTED after one pass
```

⚠ **Gotcha:** once consumed, a generator is done — you can't rewind or reuse it. To iterate twice, recreate it or use a list. This surprises people who pass a generator to two functions.

2. Iterators vs iterables — the underlying protocol

```
# An ITERABLE has __iter__ (can be looped) – list, dict, str, etc.
# An ITERATOR has __next__ (produces values one at a time)

nums = [1, 2, 3]          # iterable
it = iter(nums)           # get an iterator from it
next(it)                  # 1
next(it)                  # 2
```

Every `for` loop secretly calls `iter()` to get an iterator, then `next()` repeatedly until `StopIteration`. Generators are just a convenient way to create iterators without writing a class.

3. Building an iterator the manual way (what generators save you from)

```
# The long way – a class with __iter__ and __next__
class Countdown:
    def __init__(self, start):
        self.current = start
    def __iter__(self):
        return self
    def __next__(self):
        if self.current <= 0:
            raise StopIteration
        self.current -= 1
        return self.current + 1

# The generator way – same thing, 3 lines
def countdown(start):
    while start > 0:
        yield start
        start -= 1
```

Generators give you iterator behavior for free — no class, no `__next__`, no manual `StopIteration`.

4. `yield from` — delegate to another generator

```
def chain(*iterables):
    for it in iterables:
        yield from it          # yields everything from each iterable

list(chain([1, 2], [3, 4]))    # [1, 2, 3, 4]
```

`yield from` flattens/delegates — cleaner than a nested loop of `yield` s.

5. Infinite generators — impossible with lists

```
def infinite_counter():
    n = 0
    while True:
        yield n
        n += 1

gen = infinite_counter()
next(gen)    # 0
next(gen)    # 1
# runs forever, but only computes what you ask for — no memory blowup
```

You can't make an infinite list, but an infinite generator is fine because it's lazy. Pair with `itertools.islice` to take a slice.

6. The AI connection — streaming LLM responses

```
# This is how streaming chat responses work under the hood
def stream_response(prompt):
    for token in llm.generate(prompt):    # tokens arrive one at a time
        yield token                      # yield each as it comes

for token in stream_response("Hello"):
    print(token, end="", flush=True)
```

✅ **For your AI work:** every time ChatGPT/Claude "types" a response word by word, that's a generator streaming tokens. In FastAPI you'll return generators for Server-Sent Events (SSE) to stream LLM output — directly relevant to MSG2AI.

7. `itertools` — the generator toolkit

```
import itertools

itertools.count(10)          # 10, 11, 12... infinite
itertools.cycle([1, 2, 3])   # 1,2,3,1,2,3... infinite loop
```

```
itertools.islice(gen, 5)          # take first 5 from any generator
itertools.chain([1,2], [3,4])    # flatten iterables
```

`itertools` is a standard-library goldmine of memory-efficient generator tools.

Interview Q&A Cards

Q: `return` vs `yield` ?

A: `return` exits the function with one value. `yield` produces a value and pauses, resuming on the next request while keeping local state — making the function a lazy generator.

Q: Why use a generator over a list?

A: Constant memory — values are produced lazily, one at a time, instead of all at once. Essential for huge files, infinite sequences, and streaming.

Q: What's the single-use gotcha?

A: A generator is exhausted after one full pass; a second iteration yields nothing. Recreate it or materialize to a list if you need multiple passes.

Q: Iterable vs iterator?

A: An iterable implements `__iter__` (can produce an iterator); an iterator implements `__next__` (produces values and raises `StopIteration`). Generators are iterators created via `yield`.

 **Memory hook:** `yield` pauses and resumes, producing one value at a time. Lazy = low memory. Single-use — consume once. `()` = generator, `[]` = list. Streaming LLM tokens = generators.

✓ **One-liner for interviews:** "A generator uses `yield` to produce values lazily, one at a time, pausing and resuming while keeping local state — unlike `return` which exits. This gives constant memory for huge or infinite sequences. Generators are single-use iterators. Use generator expressions `(...)` for lazy comprehensions, and generators to stream data like LLM tokens over SSE."